```
def generateTableColumns(self,names):
    outputName = names[-1]
    inputNames = names[:-1]
    for name in inputNames:
        newInput = et.SubElement(self.decisionTableElement, "{%s}input"%(self.namespace), attrib= {"id": self.idGen("input_")})
        newInputExpression = et.SubElement(newInput, "{%s}inputExpression"%(self.namespace), attrib={"id": self.idGen("inputExpression_"),"typeRef":"double"})
        newText = et.SubElement(newInputExpression, "{%s}text"%(self.namespace))
        newText.text = name
    et.SubElement(self.decisionTableElement, "{%s}output"%(self.namespace), attrib={"id": self.idGen("output_"),"name": outputName,"typeRef":"string"})

def generateTableRows(self, mlDict):
    for k, v in mlDict:
        newRule = et.SubElement(self.decisionTableElement, "{%s}rule"%(self.namespace), attrib={"id":self.idGen("DecisionRule_")})
        for keyValues in v:
            tSignList = list(v[keyValues].keys())
            if len(v[keyValues]) == 0:
                self.createRuleCell(newRule)
            if len(v[keyValues]) == 1:
                self.createRuleCell(newRule).text = "{} {:.4f}".format(tSignList[0],v[keyValues][tSignList[0]])
            if len(v[keyValues]) == 2:
                if tSignList[0] == "<=":
                    if v[keyValues][tSignList[0]] > v[keyValues][tSignList[1]]:
                        self.createRuleCell(newRule).text = "]{:.4f}..{:.4f}]".format(v[keyValues][tSignList[1]],v[keyValues][tSignList[0]])
                    else:
                        self.createRuleCell(newRule)
                if tSignList[0] == ">":
                    if v[keyValues][tSignList[0]] < v[keyValues][tSignList[1]]:
                        self.createRuleCell(newRule).text = "]{:.4f}..{:.4f}]".format(v[keyValues][tSignList[0]],v[keyValues][tSignList[1]])
                    else:
                        self.createRuleCell(newRule)
        newOutEntry = et.SubElement(newRule, "{%s}outputEntry"%(self.namespace), attrib={"id":self.idGen("LiteralExpression_")})
        newOutText = et.SubElement(newOutEntry, "{%s}text"%(self.namespace))
        newOutText.text = str(k)
```